SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

CO1-AT2-Constraint-Based Problem Solving

Register Number	192412093
Name	Naresh Kanna. C. M

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I Naresh Kanna. C. M (192412093) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5
Application of Concepts	Effectively applies appropriate concepts, methods,	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6

	or tools				
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

Question 1: Doctor–Shift Assignment Using Backtracking Search

1. Problem statement

A hospital needs to assign three doctors (D1, D2, D3) to three shifts: Morning, Afternoon, and Night.
The assignment must satisfy these constraints:

1. D1 cannot work the Night shift.
2. D2 must work before D3.
3. Only one doctor can work each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

2. Problem formulation

Variables: D1, D2, D3

Domain: Each doctor can be assigned one of the following shifts:

$$\text{Domain}=\{M,A,N\}$$

Where:

- M = Morning
- A = Afternoon
- N = Night

Constraints:

- $D1 \neq N$
- $D3 \neq M$
- Position(D2) < Position(D3)
- No two doctors can have the same shift.
- Every doctor must have exactly one shift.

The shift order is:

$$M < A < N$$

3. Backtracking Search algorithm

Backtracking Search assigns a shift to one doctor at a time. After each assignment, it checks whether the constraints are satisfied.
If an assignment violates a constraint, the algorithm rejects it and tries another shift.

Pseudocode

Algorithm: Doctor Shift Assignment Using Backtracking

```
START

Doctors = [D1, D2, D3]
Shifts = [Morning, Afternoon, Night]

FUNCTION Backtrack(assignment):

   IF all doctors are assigned:
      RETURN assignment

   Select the next unassigned doctor D

   FOR each shift S in Shifts:

      Assign D = S

      IF all constraints are satisfied:

         result = Backtrack(assignment)

         IF result is successful:
            RETURN result

      Remove the assignment of D
```

RETURN FAILURE

Call Backtrack(empty assignment)

Print the valid schedule

STOP

4. Step-by-step backtracking with constraint checks

We assign the doctors in the order D1 → D2 → D3 and try the shifts in the order Morning → Afternoon → Night.

Step 1: Assign D1

Try D1 = Morning.

- D1 cannot work Night: satisfied.
- No other doctor has been assigned Morning: satisfied.

Result: Accept D1 = Morning.

Current assignment:

Doctor	Shift
D1	Morning
D2	Unassigned
D3	Unassigned

Step 2: Assign D2

Try D2 = Morning.

- D1 already occupies Morning.
- Only one doctor is allowed per shift.

Result: Reject D2 = Morning. Backtrack and try another shift.

Now try D2 = Afternoon.

- Afternoon is unoccupied: satisfied.
- D2 must work before D3. D3 is not assigned yet, so this constraint will be checked when D3 is assigned.

Result: Accept D2 = Afternoon.

Current assignment:

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Unassigned

Step 3: Assign D3

Try D3 = Morning.

- D3 cannot work Morning.
- Morning is already occupied by D1.

Result: Reject D3 = Morning.

Try D3 = Afternoon.

- Afternoon is already occupied by D2.

Result: Reject D3 = Afternoon.

Try D3 = Night.

- D3 cannot work Morning: satisfied.
- Night is unoccupied: satisfied.
- D2 works Afternoon and D3 works Night, so D2 works before D3: satisfied.

Result: Accept D3 = Night.

All doctors have now been assigned.

5. Final constraint verification

Constraint	Assignment	Status
D1 cannot work Night	D1 = Morning	Satisfied
D2 must work before D3	Afternoon < Night	Satisfied
Only one doctor per shift	All shifts are different	Satisfied
D3 cannot work Morning	D3 = Night	Satisfied

Constraint	Assignment	Status
Every doctor gets exactly one shift	All three doctors are assigned	Satisfied

Final valid schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

$$\boxed{D1=M,\quad D2=A,\quad D3=N}$$

Conclusion: Backtracking Search finds a valid schedule by assigning shifts one at a time, checking the constraints, and rejecting conflicting assignments.

Question 2: Robot Navigation in a 5 × 5 Grid Using Search

1. Problem statement

A robot must move from the Start (S) to the Goal (G) in a 5 × 5 grid.

The constraints are:

1. The robot can move Up, Down, Left, or Right.
2. Each movement costs 1.
3. Obstacles cannot be crossed.
4. Manhattan Distance is the given heuristic.
5. The objective is to find an optimal path.

Important: Your question does not specify the positions of S, G, or the obstacles. Therefore, the following is an illustrative example using an assumed grid. The final path may differ if your original question includes a different grid.

2. Assumed grid environment

Let S be at (1,1), G at (5,5), and the obstacles be at (2,2), (2,3), (3,3), and (4,3).

```
      Column
       1  2  3  4  5
    +---------------
Row 1 | S  .  .  .  .
Row 2 | .  X  X  .  .
Row 3 | .  .  X  .  .
Row 4 | .  .  X  .  .
Row 5 | .  .  .  .  G
```

Legend:

- S = Start
- G = Goal
- X = Obstacle
- . = Free cell

3. Manhattan Distance heuristic

Manhattan Distance estimates the distance between the current cell and the goal.

The formula is:

$$\boxed{h(n)=|x_n-x_G|+|y_n-y_G|}$$

Where:

- (x_n,y_n) are the current cell coordinates.
- (x_G,y_G) are the goal coordinates.

For the starting cell S = (1,1) and goal G = (5,5):

$$h(S)=|1-5|+|1-5|$$$$\boxed{h(S)=8}$$

Therefore, at least eight moves are required to reach the goal in this grid.

4. BFS search algorithm

Breadth-First Search (BFS) explores nodes level by level. It explores all reachable nodes at distance 1 before distance 2, and so on.

Because every move has the same cost of 1, BFS finds a shortest path.

Pseudocode

Algorithm: Breadth-First Search

START

Create an empty FIFO queue Q
Insert Start into Q

Mark Start as visited
Store the parent of Start as NULL

WHILE Q is not empty:

Remove the front node N from Q

IF N is Goal:
Reconstruct the path using parent pointers
Calculate the path cost
Return path and cost

FOR each neighbor M of N:

IF M is inside the grid
AND M is not an obstacle
AND M is not visited:

Mark M as visited
Store parent[M] = N
Insert M into Q

RETURN FAILURE

STOP

5. Step-by-step node expansion and queue updates

For this example, neighbors are checked in the order **Up → Down → Left → Right**. The queue is FIFO, meaning the first node inserted is the first node removed.
Coordinates are written as (row, column).

Step 1: Initialize

Insert S = (1,1).
Queue:
$$ Q=[(1,1)] $$
The robot expands S.
Its valid neighbors are:

- (2,1): Down
- (1,2): Right

Queue after expansion:
$$ Q=[(2,1),(1,2)] $$
Both nodes have a path cost of 1.

Step 2: Expand (2,1)

Its valid, unvisited neighbors are:

- (3,1)
- (2,2) is an obstacle.
- (1,1) is already visited.

Queue:
$$ Q=[(1,2),(3,1)] $$

Step 3: Expand (1,2)

Its valid, unvisited neighbors are:

- (1,3)
- (2,2) is an obstacle.
- (1,1) is already visited.

Queue:
$$ Q=[(3,1),(1,3)] $$
Step 4: Expand (3,1)
Its valid, unvisited neighbor is (4,1).
Queue:
$$ Q=[(1,3),(4,1)] $$
Step 5: Expand (1,3)
Its valid, unvisited neighbor is (1,4).
Queue:
$$ Q=[(4,1),(1,4)] $$
Step 6: Expand (4,1)
Its valid, unvisited neighbor is (5,1).
Queue:
$$ Q=[(1,4),(5,1)] $$
Step 7: Expand (1,4)
Its valid, unvisited neighbors are:

- (1,5)
- (2,4)

Queue:
$$ Q=[(5,1),(1,5),(2,4)] $$
Step 8: Expand (5,1)
Its valid, unvisited neighbor is (5,2).
Queue:
$$ Q=[(1,5),(2,4),(5,2)] $$
Step 9: Expand (1,5)
Its valid, unvisited neighbor is (2,5).
Queue:
$$ Q=[(2,4),(5,2),(2,5)] $$
Step 10: Expand (2,4)
Its valid, unvisited neighbors include (3,4). The cells (2,3) and (2,5) are either obstacles or already discovered as applicable.
Queue:
$$ Q=[(5,2),(2,5),(3,4)] $$
Step 11: Expand (5,2)
Its valid, unvisited neighbor is (5,3).
Queue:
$$ Q=[(2,5),(3,4),(5,3)] $$
Step 12: Expand (2,5)
Its valid, unvisited neighbor is (3,5).
Queue:
$$ Q=[(3,4),(5,3),(3,5)] $$
Step 13: Expand (3,4)
Its valid, unvisited neighbor is (4,4).
Queue:
$$ Q=[(5,3),(3,5),(4,4)] $$
Step 14: Expand (5,3)
Its valid, unvisited neighbor is (5,4).
Queue:
$$ Q=[(3,5),(4,4),(5,4)] $$
Step 15: Expand (3,5)
Its valid, unvisited neighbor is (4,5).
Queue:
$$ Q=[(4,4),(5,4),(4,5)] $$
Step 16: Expand (4,4)
Its valid, unvisited neighbors include (4,5) and (5,4), which have already been discovered.
Queue:

\[Q=[(5,4),(4,5)] \]
Step 17: Expand (5,4)
Its valid, unvisited neighbor is the Goal G = (5,5).
Queue:
\[Q=[(4,5),(5,5)] \]
Step 18: Expand (4,5)
The Goal has already been discovered, so no new node is added.
Queue:
\[Q=[(5,5)] \]
Step 19: Expand Goal G
Remove G = (5,5) from the queue.
The goal has been reached. Reconstruct the path using the stored parent pointers.
6. Optimal path and cost
One shortest path is:
\[(1,1)\rightarrow(1,2)\rightarrow(1,3)\rightarrow(1,4) \]\[\rightarrow(1,5)\rightarrow(2,5)\rightarrow(3,5) \]\[\rightarrow(4,5)\rightarrow(5,5) \]
Grid representation:

```
     1  2  3  4  5
   +---------------
Row 1 | S  →  →  →  →
Row 2 | .  X  X  .  ↓
Row 3 | .  .  X  .  ↓
Row 4 | .  .  X  .  ↓
Row 5 | .  .  .  .  G
```

There are eight movements.
\[\text{Path Cost}=\text{Number of moves}\times\text{Cost per move} \]\[=8\times1 \]\[\boxed{\text{Optimal Path Cost}=8} \]
Note about BFS and Manhattan Distance
BFS uses a FIFO queue and does not use a heuristic to prioritize nodes. Manhattan Distance is commonly used in A* Search, which selects nodes using:
\[\boxed{f(n)=g(n)+h(n)} \]
Here, \(g(n)\) is the cost so far, and \(h(n)\) is the estimated remaining cost.
Since your question specifically asks for BFS, the worked example uses BFS. The Manhattan Distance formula is included separately as required.
Conclusion: BFS finds a shortest path of cost 8 for the assumed grid because all movements have equal cost and obstacles are avoided.

Question 3: Autonomous Rescue Robot in a Disaster-Affected Building
a) Analyze the constraints and explain their effect on decision-making
An autonomous rescue robot must locate a survivor in a disaster-affected building. It starts at S and must reach G while avoiding obstacles and risky areas.
The robot has limited sensing capability and can observe only adjacent cells. Therefore, it cannot know the entire building layout in advance.
1. Movement constraints
The robot can move only in four directions:

- Up
- Down
- Left
- Right

Effect: The robot cannot move diagonally. It must find a valid sequence of horizontal and vertical movements.
2. Obstacle constraints
Some cells contain obstacles and cannot be traversed.
Effect: The robot must identify blocked cells and avoid them. If a route is blocked, it must search for another available route.
3. Limited sensing capability
The robot can observe only adjacent cells.

Effect: It cannot rely on a complete map at the beginning. It must explore its surroundings and update its knowledge as it moves.

4. Movement cost

Each movement has a cost of 1.

Effect: The robot should avoid unnecessary movements and choose a shorter route when other conditions are equal.

5. Risky-zone cost

Entering a risky zone adds an extra cost of 2.

Therefore:

$$\text{Normal movement cost}=1$$
$$\text{Risky movement cost}=1+2=3$$

Effect: The robot should consider both distance and risk. A longer safe route may be preferable to a shorter route containing several risky zones.

6. Dynamic knowledge updating

The robot must update its knowledge of the environment while exploring.

Effect: When it discovers an obstacle or a risky area, it must update its internal map and reconsider its planned route if necessary.

Overall objective

The robot must minimize total travel cost while avoiding obstacles and maintaining safe navigation.

$$\boxed{\text{Total Cost}=\text{Movement Costs}+\text{Additional Risk Costs}}$$

---

b) Develop a solution using Uniform Cost Search and an Online Search Agent

1. Why choose Uniform Cost Search?

Uniform Cost Search (UCS) expands the available path with the lowest cumulative cost.

It considers both ordinary movement costs and additional risk costs.

For every move:

$$g(n)=g(\text{previous node})+\text{cost of the new move}$$

Where $g(n)$ is the cumulative cost of reaching node n.

For a normal cell:

$$g(n)=g(\text{previous node})+1$$

For a risky cell:

$$g(n)=g(\text{previous node})+3$$

The robot uses a priority queue to select the path with the smallest cumulative cost.

2. Algorithm

Algorithm: Online Rescue Robot Using UCS

START

Initialize the robot at position S

Create a known map containing the starting cell
Create an empty priority queue PQ
Insert (S, 0) into PQ

WHILE the goal has not been reached:

  Observe the adjacent cells

  Update the known map with:
    Free cells
    Blocked cells
    Risky cells

  Update the available routes

  IF no safe reachable route is available:
    Report that the goal cannot currently be reached
    STOP

```
    END IF

    Select the reachable frontier node
    with the lowest estimated cumulative cost

    IF the selected node is blocked:
        Reject that route
        Update the map
        CONTINUE
    END IF

    Calculate movement cost:

        Normal cell = 1
        Risky cell = 3

    Update the cumulative path cost

    Move to the selected cell

    IF the current position is G:
        Stop the search
        Report successful rescue
        Report the total path cost
    END IF

END WHILE

STOP
```

Implementation note: The pseudocode illustrates the online decision process. A complete implementation should maintain parent pointers, track the lowest known cost for each cell, and replan whenever new observations invalidate the current route. For guaranteed optimality, UCS requires a correct cost model and sufficient exploration of the reachable environment.

3. Steps involved

Step 1: Initialize

The robot starts at S and records its current location.

Step 2: Sense the surroundings

The robot observes the four adjacent cells and identifies which cells are free, blocked, or risky.

Step 3: Update the map

The robot adds the new observations to its internal map.

Step 4: Calculate movement costs

Each ordinary movement costs 1. Entering a risky zone costs 3.

Step 5: Select the lowest-cost route

The robot compares the cumulative costs of the known routes and selects the lowest-cost option.

Step 6: Move and observe again

After moving, the robot senses the adjacent cells again and updates its map.

Step 7: Replan when necessary

If the robot discovers an obstacle or a new risky area, it updates the route and recalculates the costs.

Step 8: Reach the survivor

When the robot reaches G, it stops and reports the final route and total cost.

4. Illustrative cost comparison

Consider two hypothetical routes from S to G.

Route	Normal moves	Risky-zone entries	Total cost
Route A	6	2	\(6+(2\times3)=12\)
Route B	8	0	\(8+(0\times3)=8\)

For Route A:
$$C_A=6+(2\times3)=12$$
For Route B:
$$C_B=8+(0\times3)=8$$
Since:
$$8<12$$
The robot selects **Route B**, provided it is reachable and safe.
This example shows that the shortest route is not necessarily the least-cost route.
These route costs are illustrative; the question does not provide a specific building grid or the locations of risky cells.

---

c) Apply AI concepts

1. Intelligent agent

An intelligent agent perceives its environment and takes actions to achieve a goal.
The rescue robot is an intelligent agent because it:

- Observes surrounding cells using sensors.
- Identifies obstacles and risky areas.
- Selects movements based on available information.
- Updates its knowledge of the building.
- Plans a route to reach the survivor.

The robot follows the general agent cycle:
$$\boxed{\text{Perceive}\rightarrow\text{Decide}\rightarrow\text{Act}}$$
It repeats this cycle until it reaches the goal or determines that the goal cannot currently be reached.

2. Rationality

A rational agent selects actions expected to maximize its performance measure, based on its available information.
For the rescue robot, the performance measures include:

1. Reaching the survivor.
2. Minimizing total travel cost.
3. Avoiding obstacles.
4. Reducing exposure to risky zones.
5. Using available information effectively.

A rational robot does not simply choose the nearest cell. It considers the cumulative cost and the safety of the available route.
For example, it may select a longer route with no risky cells instead of a shorter route with high risk costs.

3. Environment type

The rescue robot operates in the following environment:
Partially observable: The robot sees only adjacent cells, not the entire building.
Sequential: Each movement affects the robot's future choices and available routes.
Dynamic: New obstacles or hazards may be discovered during exploration, requiring the robot to update its plan.
Stochastic or uncertain: Movement outcomes and environmental conditions may be uncertain. If the building is assumed to remain unchanged and movements are predictable, the environment can instead be treated as deterministic.
Discrete: The building is represented as a grid of individual cells.
Single-agent: The robot can be modeled as a single decision-making agent, although it may interact with survivors or other rescue teams.
Partially known: The robot initially lacks complete information about the building and learns more through exploration.

---

d) Justify why UCS with online search is suitable

Uniform Cost Search combined with online exploration is appropriate for this rescue problem for the following reasons.

1. Considers different movement costs

UCS considers the actual cost of each movement, including additional risk costs.
It can distinguish between a normal movement costing 1 and a risky movement costing 3.

2. Finds a least-cost route

When the map and movement costs are fully known, UCS finds an optimal route if all step costs are non-negative.

With an initially incomplete map, the robot must explore and update its knowledge before it can guarantee that the route is globally optimal.

3. Supports dynamic knowledge updating

An online search agent observes the environment while acting. It can update its map and replan when it discovers new obstacles or risky areas.

4. Avoids unnecessary risk

By including additional risk costs, UCS can prefer a safer route even when it requires more movements.

However, assigning a higher cost to risky cells is a simplified risk model. Real rescue robots may also need hard safety restrictions that prevent entering certain dangerous cells altogether.

5. Suitable for partially observable environments

The robot cannot initially see the entire building. Online exploration allows it to make decisions using its current knowledge and improve its plan as new information becomes available.

Conclusion

Uniform Cost Search combined with an online search agent is suitable for the autonomous rescue robot because it considers cumulative travel costs, accounts for risky zones, and supports replanning as the robot learns more about the environment. It helps the robot search for a low-cost route while avoiding known obstacles and respecting safety constraints.